LUMS
A Not-for-Profit University

School of Science and Engineering

# AI 501 Mathematics for Artificial Intelligence

# ASSIGNMENT 3

---

**Due Date:** 1 pm, November 30, 2025
**Format:** 10 problems for a total of 100 marks
**Instructions:**

- You are allowed to collaborate with your peers but copying your colleague's solution is strictly prohibited. This is not a group assignment. Each student must submit his/her own assignment.
- Solve the assignment on blank A4 sheets and staple them before submitting.
- **There may be more than one correct solution for non-numeric questions.**
- Submit in-class or in the dropbox labeled AI-501 outside the instructor's office.
- **Write your name and roll no. on the first page.**
- Feel free to contact the instructor or the teaching assistants if you have any concerns.

- You represent the most competent individuals in the country, do not let plagiarism come in between your learning. In case any instance of plagiarism is detected, the disciplinary case will be dealt with according to the university's rules and regulations.
- We require you to acknowledge any use or contributions from generative AI tools. Include the following statement to acknowledge the use of AI where applicable.

  *I have used [insert Tool Name] to [write, generate, plot or compute; explain specific use of generative AI] [number of times].*

---

**Problem 1** (8 marks)

In the depths of the Anthropic Research Laboratory, Dr. Elena Chen discovers that the attention mechanism of a large language model can be represented as a Taylor polynomial. The model, codenamed Turing_$n$, processes text through layers where $\mu$ and $\sigma$ represent learnable parameters.

Compute the Taylor polynomial $T_n$ with $n = 0, \ldots, 5$ coefficients of the consciousness function $f(x) = \sin(x) + \cos(x)$ at the critical point $x = 0$. This represents how the model approximates its activation functions during inference.

**Problem 2** (8 marks)

Three neural architects are tasked with designing transformation functions for the embedding space of a next-generation language model. Each function maps tokens to their semantic representations.

Consider the following embedding transformations and determine the dimensional structure of each transformation's output space:

Transformation Alpha (Logarithmic Encoder):

$$f_1(x) = \sin(x_1)\cos(x_2), \quad x \in \mathbb{R}^2$$

Transformation Beta (Weighted Projection):

$$f_2(x, y) = x^\top \cdot y, \quad x, y \in \mathbb{R}^n$$

Transformation Gamma (Exponential Attention):

$$f_3(x) = xx^\top, \quad x \in \mathbb{R}^n$$

(a) What are the dimensions of $\frac{\partial f_1}{\partial x}$?

(b) Compute the Jacobians that govern gradient flow through this architecture.

**Problem 3** (8 marks)

During backpropagation through a transformer's attention layers, the gradient flow encounters a nested computation path. The forward pass computes:

$$f(t) = \sin(log(t^\top \cdot t)), \quad t \in \mathbb{R}^D$$

The attention matrix is defined as:

$$a(X) = \text{tr}(AXB)$$

Differentiate $f$ with respect to $t$ and $a$ with respect to $X$.

**Problem 4** (8 marks)

A research team at DeepMind is experimenting with novel activation functions. They need to compute all partial derivatives to understand the gradient landscape during training.

Compute the derivatives $\partial f / \partial x$ of the following activation candidates by using the chain rule. Provide the dimensions of every single partial derivative. Describe your steps in detail.

Activation Candidate A (Logarithmic Softplus):

$$f(z) = \log(1 + z), \quad z = x^\top x, \quad x \in \mathbb{R}^D$$

Activation Candidate B (Generalized Sine Activation):

$$f(z) = \sin(z), \quad z = Ax + b, \quad A \in \mathbb{R}^{E \times D}, b \in \mathbb{R}^E, x \in \mathbb{R}^D$$

where $\sin(\cdot)$ is applied to every element of $z$.

**Problem 5** (8 marks)

An AI safety researcher is analyzing the loss surface of a language model to understand training stability. The model's loss function has a complex geometric structure.

The loss function for token prediction is:

$$f(z) = \exp\left(-\frac{1}{2}z\right)$$

The pre-activation logits are computed as:

$$z = g(y) = y^\top S^{-1} y$$

The hidden representation is:

$$y = h(x) = x - \mu$$

where $\mu, x \in \mathbb{R}^D$, $S \in \mathbb{R}^{D \times D}$.

Compute the derivatives $\partial f / \partial x$ using the chain rule. Provide the dimensions of every single partial derivative. Describe your steps in detail.

**Problem 6** (8 marks)

You are an AI researcher analyzing the loss surface of a neural network. The model's loss function over a single parameter space is given by:

$$f(x) = x^3 + 6x^2 - 3x - 5$$

This represents a simplified view of how loss changes as we adjust one critical hyperparameter during training.

Find the stationary points of this loss landscape and classify each one. Determine whether they represent:

- Local minima (stable training points where the model converges)
- Local maxima (unstable peaks to avoid)
- Saddle points (treacherous plateaus that slow down training)

**Problem 7** (8 marks)

A team at DeepMind is investigating the geometric properties of feasible regions in their optimization problems. They've encountered three controversial claims about convex sets.

Determine whether the following statements are TRUE or FALSE. Provide rigorous proofs or counterexamples.

(a) The Intersection Theorem: "The intersection of any two convex sets is convex."

(b) The Union Conjecture: "The union of any two convex sets is convex."

(c) The Difference Dilemma: "The difference of a convex set $A$ from another convex set $B$ is convex."

**Problem 8** (12 marks)

The ML operations team at Cohere is training a large classification model using the hinge loss (the loss function used by support vector machines):

$$L(\alpha) = \max(0, 1 - \alpha)$$

However, they want to apply gradient-based methods like L-BFGS that require smooth, differentiable objectives. The problem? The hinge loss has a kink at $\alpha = 1$.

(a) Compute the convex conjugate of the hinge loss $L^*(\beta)$, where $\beta$ is the dual variable. Add a proximal term and compute the conjugate of:

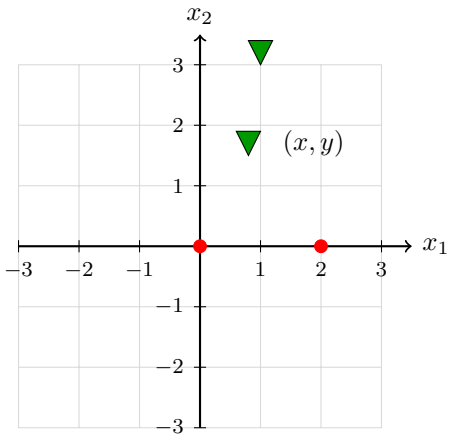
$$L^*(\beta) + \frac{\gamma}{2}\beta^2$$

where $\gamma$ is a given hyperparameter that controls smoothing strength.

(b) Explain how this transformation enables the use of subgradient-free methods while approximating the original hinge loss.

**Problem 9** (16 marks)

Consider the following 2-class classification problem with circle class as the positive class and triangle class as the negative class. There are a total of 4 points; however, one of the points is labeled as $(x, y)$.




(a) Determine the range of values for $x$ and $y$ for which the data remains linearly separable.

(b) Now suppose $x = y = 1$:

(i) Draw the maximum margin separator on the graph.

(ii) Clearly label Support Vectors.

(iii) Find bias $w_0$ and orientation vector $w = [w_1, w_2]$ for the maximum margin separator.

(c) Now suppose $x = 1$ and $y = 4$:

(i) Draw the maximum margin separator on the graph.

(ii) Clearly label Support Vectors.

(iii) Find bias $w_0$ and orientation vector $w = [w_1, w_2]$ for the maximum margin separator.

**Problem 10** (16 marks)

In the following figure, a 2-class (circle ($-1$) or triangle ($+1$)) classification is being considered based on two input features $x_1$ and $x_2$. Clearly, the data is not linearly separable by a single linear classifier. Our goal is to obtain a new representation of the original 2-dimensional data such that the new data becomes linearly separable by a single linear classifier in the new domain. For this purpose, perform the following:

(a) Using the given data points, fit a linear classifier of the form:

$$w^T x + w_0 = 0$$

using the average Hinge-loss over all points:

$$I(w) = \frac{1}{N} \sum_{i=1}^{N} \max(0, 1 - m_i)$$

where $m_i = y_i(w^T x_i + w_0)$ is the margin.

You may want to write a small piece of code for Gradient Descent to obtain the final weight vector. Report the training accuracy of this classifier (specifically, how many points are misclassified?).

(b) Now, use the following feature map (or kernel) to transform every 2-dimensional point $x_i$ into a new 3-dimensional vector $\hat{\mathbf{x}}_i$:

$$\hat{\mathbf{x}}_i = \begin{bmatrix} x_{i1}^2 \\ x_{i2}^2 \\ \sqrt{2} x_{i1} x_{i2} \end{bmatrix}$$

Once you have the new data, fit a linear classifier (with one extra dimension) on the transformed data:

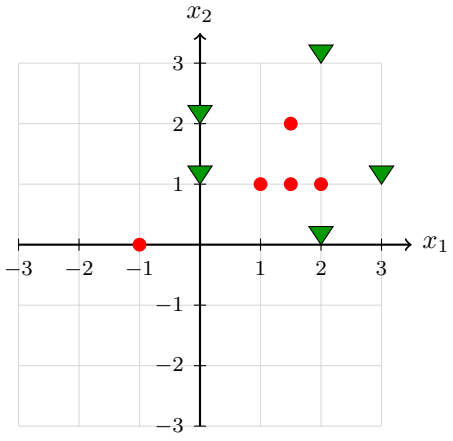
$$w^T \hat{\mathbf{x}} + w_0 = 0$$

using the average Hinge-loss over all points:

$$\tilde{I}(w) = \frac{1}{N} \sum_{i=1}^{N} \max(0, 1 - \hat{m}_i)$$

where $\hat{m}_i = y_i(w^T \hat{\mathbf{x}}_i + w_0)$.

You may want to write a small piece of code for Gradient Descent to obtain the final weight vector. Report the training accuracy of this classifier (specifically, how many points are misclassified?).




— End of Assignment —